# Internal Partitioning

Design and Analysis of Algorithms (DAA) — C Language

## Purpose

Partition an array in-place around a pivot as used in Quick Sort. The last element is chosen as the pivot; values less than or equal to it are moved to its left.

## C Program

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

/* In-place (internal) partition using the last element as pivot */
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\\n");
}

int main(void) {
    int arr[] = {29, 10, 14, 37, 13, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, n);

    quickSort(arr, 0, n - 1);

    printf("Sorted array:   ");
```

```
    printArray(arr, n);
    return 0;
}
```

## Sample Output

```
Original array: 29 10 14 37 13 5
Sorted array:   5 10 13 14 29 37
```